IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Bryan Wai Kit Li

April 19, 2026

Instructions and guidelines (read carefully)

Instructions

1. Insert your name as it appears on your Online Campus learner profile in the file name. Save the file as: **First Name Last Name M10U4 Capstone Project – e.g., Lilly Smith M10U4 Capstone Project.**
2. Write all your answers and upload your screenshots in this PowerPoint presentation in the slides provided.
3. Convert your final presentation into **.pdf** format and submit this **PDF document**. No other file types will be accepted.

Note: Please ensure that you have checked your course calendar for the due date for this assignment.

Guidelines

Note: If you do not have a desktop version, follow the instructions from the previous lab to access PowerPoint online.

1. Complete the slides with relevant details (you are free to add additional slides, charts, and tables).
2. Add any additional unused charts or tables to the appendix that you think are relevant to your analysis.

Plagiarism declaration
1. I understand that plagiarism is to use another's work and pretend that it is one's own. 2. I have not allowed, and will not allow, anyone to copy my work with the intention of submitting it as their own. 3. I acknowledge that copying someone else's assignment (or part of it) is wrong and declare that my assignments are my own work. **Plagiarism cases will be investigated in line with the Terms and Conditions for learners.**

Outline

- Executive summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive summary

- This project explores the feasibility of predicting the landing outcome of Falcon 9's first stage using machine learning (ML).
- Data was collected from SpaceX API and Wikipedia using Python libraries "requests" and BeautifulSoup. It was followed by data cleaning, data normalization, and feature engineering using Python.
- Exploratory data analysis (EDA) was conducted to identify ideal features for model training. Data visualization was done using Matplotlib, Folium, and Poly Dash. Summary statistics on the data was generated using SQLLite3.
- ML algorithms were trained and tested to identify the best one for prediction using Scikit-Learn (SL).
- Support Vector Machine (SVM) was identified as the best at the end.

Introduction

- SpaceY, founded by Allon Mask, plans to enter into the space travel industry.
  - It will put SpaceY in direct competition with SpaceX, founded by Elon Musk.
- One competitive advantage SpaceX has is cost.
  - It could attempt landing its Falcon 9's first stage and, if successful, can reuse the first stage.
  - It thus could possibly bid lower for projects than its competitors, if it deems the first stage can be landed successfully and reused.
- To be competitive in the bidding process with SpaceX, SpaceY wants to predict what the final ultimate cost would be for SpaceX.
  - It wants to predict the landing outcome w. Falcon 9's first stage.
  - It will then predict how SpaceX will bid.
- SpaceY wants to make such predictions using ML.

Section 1

Methodology

Methodology

Executive summary

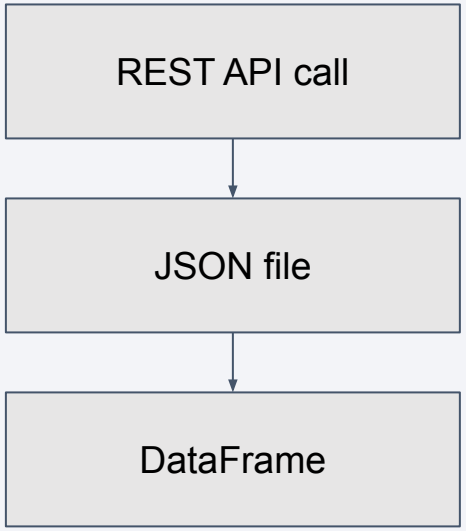
- Data collection methodology
- Perform data wrangling.
- Perform exploratory data analysis (EDA) using visualization and SQL.
- Perform interactive visual analytics using Folium and Plotly Dash.
- Perform predictive analysis using classification models.

Data collection

- Data for model development and evaluation was retrieved from SpaceX API and Wikipedia.
- API calls and web scraping were performed using Python libraries.
- Data wrangling including data cleaning, data normalization, and feature engineering was performed.

Data collection – SpaceX API

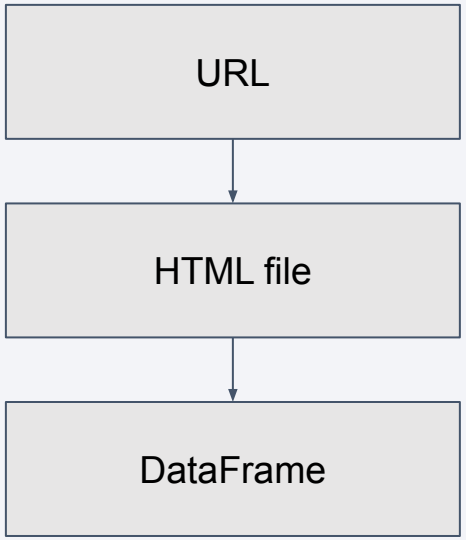
- A dataset of flight records would be retrieved from SpaceX API via REST API.
- Multi-Step process:
  1. “requests” object to send Initial HTTP request to an endpoint
     a. “requests” object to receive JSON file in initial HTTP response
     b. “requests” object to parse JSON file into Dictionary variable
  2. Dictionary variable to convert into DataFrame object
  3. DataFrame object to filter out flights w. multiple “cores” or “payloads”
  4. DataFrame object to remove unrelated fields
  5. Fields “flight_number”, “date”, and certain attributes of “cores” in DataFrame object to convert into List variables
  6. “requests” object to send addl. HTTP requests to other endpoints w. inputs extracted from initial HTTP response such as field “rocket”
     a. “requests” objects to receive JSON files in addl. HTTP responses
     b. “requests” objects to parse JSON files into Dictionary variables
  7. Dictionary variables to convert into List variables
  8. Fields from initial and addl. responses stored in List variables to convert into DataFrame object
  9. DataFrame object to filter out non-Falcon-9 flights
  10. DataFrame object to export to CSV file



Notebook’s Github link:
https://github.com/liwaikitbryan/IBM-DS-2026-01-28-Capstone/tree/main/Supporting/9.4/9.4.3

Data collection – Scraping

- A dataset of flight records would be scraped from a Wikipedia webpage.
- Multi-Step process:
  1. “requests” object to send HTTP GET request w. URL to webpage
     a. “requests” object to receive HTML file in HTTP response
  2. BeautifulSoup object to convert HTML file into tree of HTML tag objects
  3. Initialize Dictionary variable w. keys for empty List variables
  4. Iterate over “table” tag objects in BeautifulSoup object
     a. In each iteration, iterate over “tr” tag objects
        i. In each iteration, determine if “tr” tag objects relates to flight record
        ii. If it does, extract string values from associated “tr” tag objects of relevance such as “Version Booster”
        iii. Store string values in corresponding List variables of Dictionary var
  5. Dictionary variable to convert into DataFrame object
  6. DataFrame object to export to CSV file



Notebook’s Github link:
https://github.com/liwaikitbryan/IBM-DS-2026-01-28-Capstone/tree/main/Supporting/9.4/9.4.4

Data wrangling

- The collected dataset of flight records would be pre-processed before use.
- Multi-Step process:
  1. Identify fields with missing values using DataFrame
     a. Fields “PayloadMass” and “LandingPad” identified
     b. Replace missing values in field “PayloadMass” with mean from other records
     c. Leave field “LandingPad” as is, with missing value indicating no landing pad used
  2. Using DataFrame, add dummy (i.e. 1 = success; 0 = failure) field “Class”, transformed from field “outcome” encapsulating landing’s success/failure outcome and type; to use as target (i.e. Y)
  3. Using DataFrame and informed by EDA, set aside select fields such as “LaunchSite” to use as features (i.e. X)
  4. Apply one-hot-encoding to categorical targets
  5. Normalize targets using Scikit-Learn

Notebook’s Github link: https://github.com/liwaikitbryan/IBM-DS-2026-01-28-Capstone/tree/main/Supporting/9.4/9.4.8

EDA with data visualization

- Charts were created using Matplotlib to examine relationships among the features and target.
- Chart 1: Scatter plot to display trend of flights w.r.t. landing outcome and payload mass carried
  - To show over time potential clusters of landing success or failure and signal correlation between landing outcome and payload mass carried
- Chart 2: Scatter plot to display trend of flights w.r.t. landing outcome and launch site used
  - To show over time potential clusters of landing success or failure and signal correlation between landing outcome and launch site used
- Chart 3: Scatter plot to display distribution of flights' payload mass carried; segmented by launch site and dots color-coded to indicate success or failure in landing
  - To show relationship between payload mass carried and launch site used and signal potential collinearity
- Chart 4: Bar chart to display landing success rate w.r.t. orbit travelled
  - To show potential correlation between orbit travelled and landing outcome
- Chart 5: Scatter plot to display trend of flights w.r.t. landing outcome and orbit travelled
  - To show over time potential clusters of landing success or failure and signal correlation between landing outcome and orbit travelled
- Chart 6: Scatter plot to display distribution of flights' payload mass carried; segmented by orbit travelled and dots color-coded to indicate success or failure in landing
  - To show relationship between payload mass carried and orbit travelled and signal potential collinearity
- Chart 7: Line chart to display landing success rate over time
  - To show if there is trend in landing success rate
- See results in Section 2

Notebook's Github link: https://github.com/liwaikitbryan/IBM-DS-2026-01-28-Capstone/tree/main/Supporting/9.5/9.5.4

EDA with SQL

- SQL queries were run using SQLLite3 to examine flight records w. summary statistics.
- Query 1: Display all names of unique launch sites
- Query 2: Display five records where launch sites begin with string “KSC”
- Query 3: Display total payload mass carried by boosters launched by NASA (CRS)
- Query 4: Display average payload mass carried by booster version F9 v1.1
- Query 5: Display date when the successful landing outcome in drone ship was first achieved.
- Query 6: Display all names of boosters which have successful landing on ground pad and have payload mass carried > 4,000 kg but < 6,000 kg
- Query 7: Display total numbers of success and failure mission outcomes
- Query 8: Display all booster versions that have carried maximum payload mass
- Query 9: List all records from year 2017 w. successful landing in ground pad, to display month names, landing outcomes, booster versions, and launch sites
- Query 10: Rank counts of landing outcome types seen between June 4, 2010 and March 20, 2017, in descending order
- See results in Section 2

Notebook’s Github link:
https://github.com/liwaikitbryan/IBM-DS-2026-01-28-Capstone/tree/main/Supporting/9.5/9.5.2

Build an interactive map with Folium

- folium.map was used to visualize launch sites geographically.
- Task 1: For each launch site, add folium.Circle object and folium.map.Marker object
  - folium.Circle object to circle launch site w. red circle; folium.map.Marker object to display site name as red texts
  - To pinpoint locations of all launch sites on global map
- Task 2: For each flight, add to associated launch site folium.map.Marker object
  - folium.map.Marker object to display icon colored green or red to indicate landing success or failure respectively
  - folium.map.Marker to add to MarkerCluster object to expand or collapse showing of icons
  - To display launch site's landing success and failure counts
- Task 3: Add MousePosition object to display coordinate for mouse-over
  - To calculate launch site's distance from its nearby locations of interest
- Task 4: For each of nearest coastline, highway, and railroad, display the calculated distance from launch site "CCAFS SLC 40"
  - folium.PolyLine object to display a line in between and folium.Marker to display the distance in red texts
  - To assess geographical characteristics of launch site, possibly affecting landing success rate.
- See results in Section 3

Notebook's Github link: https://github.com/liwaikitbryan/IBM-DS-2026-01-28-Capstone/tree/main/Supporting/10.1/10.1.3

Build a dashboard with Plotly Dash

- Plotly Dash was used to visualize landing outcomes by launch site or payload mass carried.
- Task 1: Add Dropdown to select displaying in existing pie chart landing outcomes in aggregate or by individual launch site
- Task 2: Add callback function to support deliverables in Task 1
  - Callback function w. pie chart as output parameter and Dropdown as input parameter
  - In aggregate mode, display in pie chart distribution of success counts by launch site
  - In individual mode, display in pie chart success/failure ratio
  - To show distribution of landing successes w.r.t. launch sites and site-specific success rates
- Task 3: Add RangeSlider to specify range of payload mass carried to limit flights to include in existing scatter plot
- Task 4: Add callback function to support deliverables in Task 3
  - Callback function w. scatter plot as output parameter, Dropdown as input parameter 1, RangeSlider as input parameter 2
  - Callback function to include in scatter plot flights with payload mass carried within range specified, either across all launch sites or from one specific site
  - Scatter plot to display flights' distribution w.r.t. payload mass carried, segmented by landing success/failure outcome
  - Scatter plot to color-code flights with different colors for different booster version category
  - To show impact of launch site used, range of payload mass carried, and booster version category on landing outcome
  - See results in Section 4

py file's Github link: https://github.com/liwaikitbryan/IBM-DS-2026-01-28-Capstone/tree/main/Supporting/10.1/10.1.4

Predictive analysis (classification)

- SL was used in order to train and evaluate ML algorithms in order to identify the best estimator.
- ML algorithms to use: Logistic Regression (LR), SVM, Decision Tree (DT), and K-Nearest Neighbors (KNN)
- Multi-Step process:
  1. Convert feature and target datasets to NumPy type from DataFrame type
  2. Normalize feature dataset using SL's StandardScaler
  3. Split each of feature and target datasets into training and testing sets using SL's function "train_test_split", setting parameter "test_size" to % of data for testing
  4. Initialize SL.GridSearchCV object and run cross validation (CV) on each ML algorithm, setting parameters "param_grid" to values of hyperparameters (HP) chosen to train on, "scoring" to "accuracy", and "cv" to # of folds chosen
     a. Train ML algorithm w. training sets using GridSearchCV object's "fit" function
     b. Determine best combination of HP values based on best mean "accuracy" score across CV iterations
     c. Generate best estimator, assuming best combination of HP values and refitting to whole training set
     d. Determine best estimator's "accuracy" score on testing set
  5. Choose best out of four best estimators based on "accuracy" score on testing set
- See results in Section 5

Notebook's Github link: https://github.com/liwaikitbryan/IBM-DS-2026-01-28-Capstone/tree/main/Supporting/10.2/10.2.3

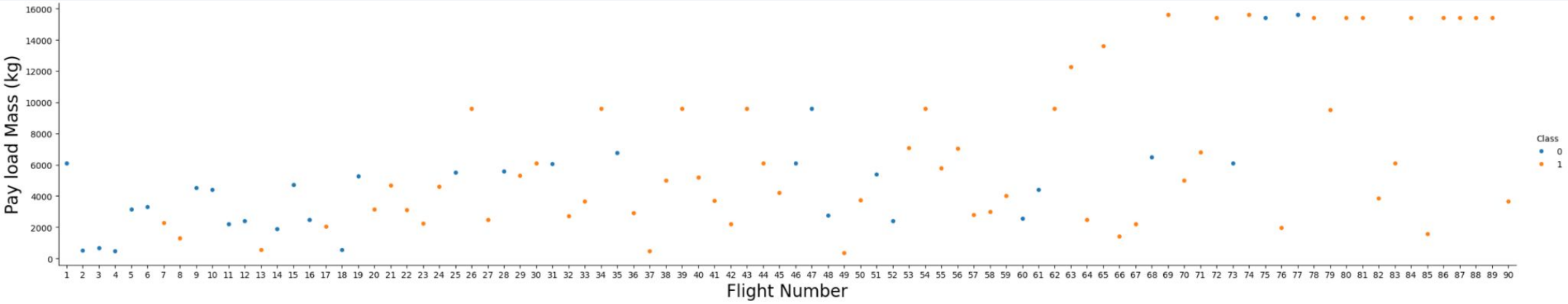
Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

Section 2

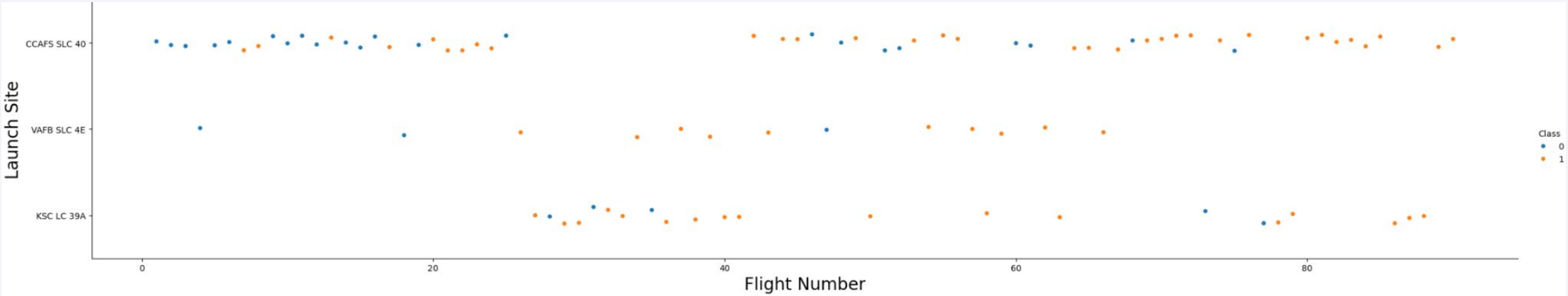
Insights drawn from EDA

Flight number vs. payload



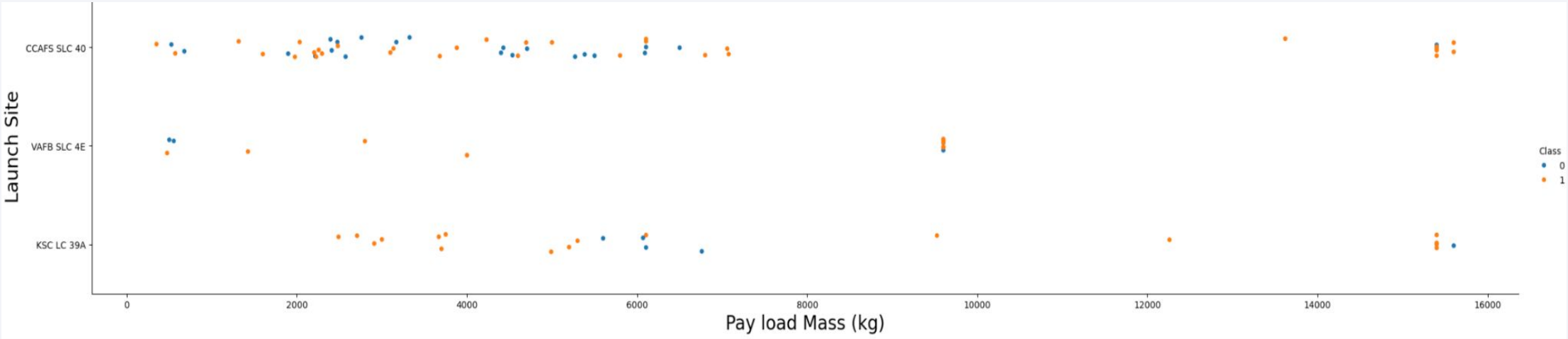
- The scatter plot displays historically flights' landing outcomes and payload mass carried.
- Flights are indexed and ordered chronologically by flight number, a consecutive sequence of integers starting at 1.
- Each dot is represented by a pair of flight number and payload mass carried and color-coded to indicate 1 = success (blue) or 0 = failure (orange) in landing.
- The more recent time had a higher success rate in landing overall.
- Payload mass carried by and large trended up, coinciding with a higher landing success rate in recent time.

Flight number vs. launch site



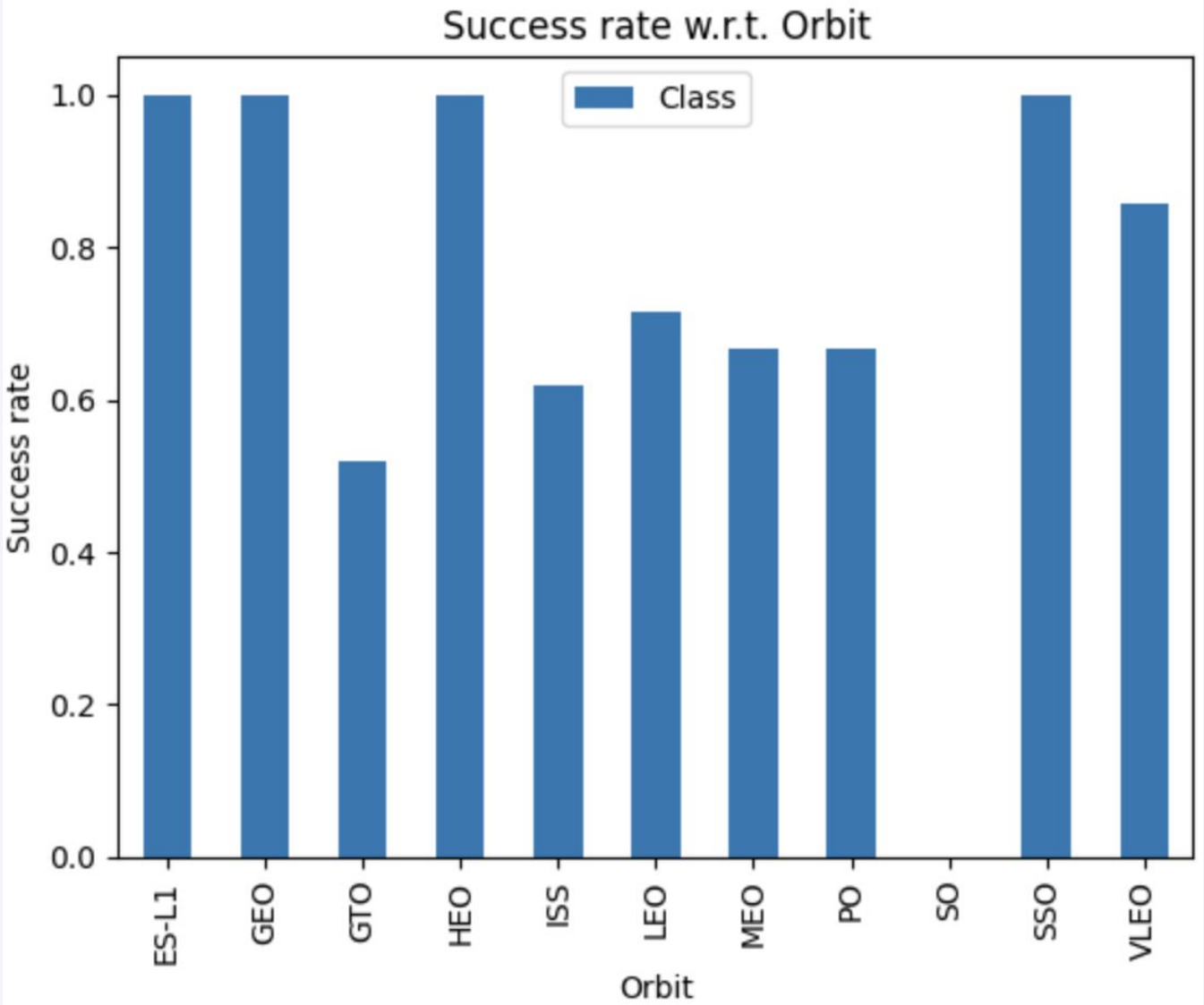
- The scatter plot displays historically flights' landing outcomes and launch sites used.
- Flights are indexed and ordered chronologically by flight number, a consecutive sequence of integers starting at 1.
- Each dot is represented by a pair of flight number and launch site used and color-coded to indicate 1 = success (blue) or 0 = failure (orange) in landing.
- The more recent time had a higher success rate in landing overall.
- The more recent time saw flights concentrating at site "SLC CCAFS40", coinciding with a higher landing success rate.

Payload vs. launch site



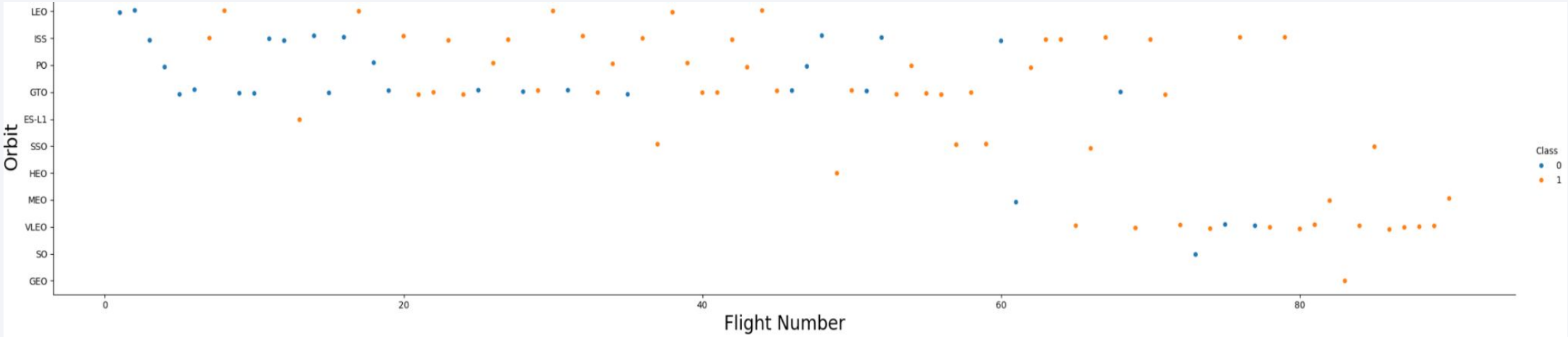
- The scatter plot displays for each launch site its flight distribution with respect to payload mass carried and color-codes each flight's landing outcome.
- The high end of the payload mass spectrum sees a higher landing success rate across all launch sites, although the conclusion could be limited by the small sample size.

Success rate vs. orbit type



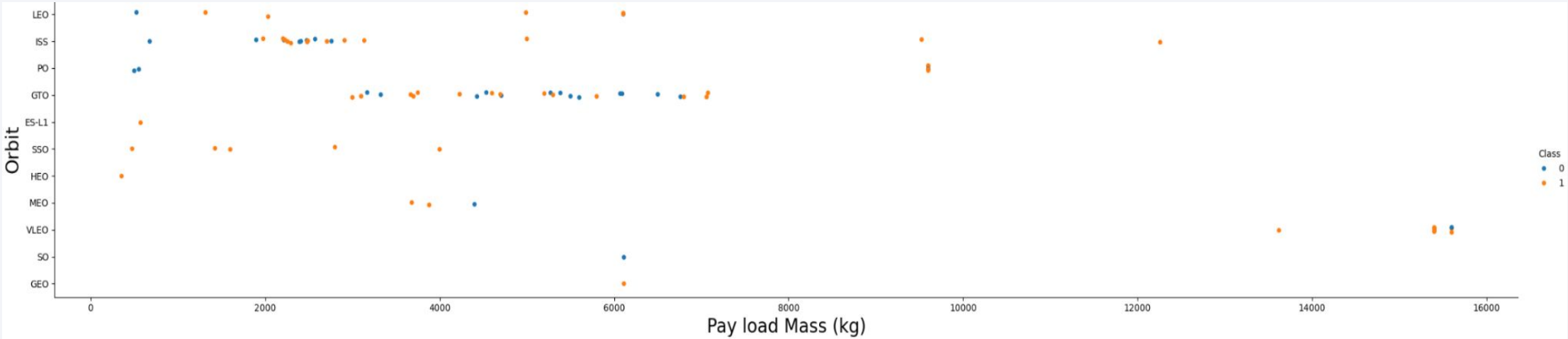
- The bar chart displays the landing success rate with respect to orbit travelled.
- Some orbits have a higher success rate than others.
- The conclusion could be limited by certain orbits' small sample sizes.

Flight number vs. orbit type



- The scatter plot displays historically flights' landing outcomes and orbits travelled.
- Flights are indexed and ordered chronologically by flight number, a consecutive sequence of integers starting at 1.
- Each dot is represented by a pair of flight number and orbit travelled and color-coded to indicate 1 = success (blue) or 0 = failure (orange) in landing.
- The more recent times had a higher success rate in landing overall.
- The more recent time saw flights concentrating at orbit "VLEO", coinciding with a higher landing success rate.

Payload vs. orbit type



- The scatter plot displays for each orbit travelled its flight distribution with respect to payload mass carried and color-codes each flight's landing outcome.
- The high end of the payload mass spectrum sees a higher landing success rate across all orbits travelled, although the conclusion could be limited by the small sample size.

Launch success yearly trend

- The line chart displays overtime the overall landing success rate.
- The upward trajectory could be attributed to higher frequencies of certain feature labels.

All launch site names

Launch_Site
CCAFS LC-40
CCAFS SLC-40
KSC LC-39A
VAFB SLC-4E

- The SQL query returns the unique set of launch site names.

Launch site names begin with “KSC”

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2017-02-19	14:39:00	F9 FT B1031.1	KSC LC-39A	SpaceX CRS-10	2490	LEO (ISS)	NASA (CRS)	Success	Success (ground pad)
2017-03-16	6:00:00	F9 FT B1030	KSC LC-39A	EchoStar 23	5600	GTO	EchoStar	Success	No attempt
2017-03-30	22:27:00	F9 FT B1021.2	KSC LC-39A	SES-10	5300	GTO	SES	Success	Success (drone ship)
2017-05-01	11:15:00	F9 FT B1032.1	KSC LC-39A	NROL-76	5300	LEO	NRO	Success	Success (ground pad)
2017-05-15	23:21:00	F9 FT B1034	KSC LC-39A	Inmarsat-5 F4	6070	GTO	Inmarsat	Success	No attempt

- The SQL query returns the list of flights with their launch site’s name beginning with “KSC”.

Total payload mass

Customer	Total_Payload_Mass
NASA (CRS)	45596

- The SQL query returns the aggregate payload mass carried for customer “NASA (CRS)”, which is equal to 45,596 kg.

Average payload mass by F9 v1.1

Booster_Version	Avg_Payload_Mass
F9 v1.1	2928.4

- The SQL query returns the average payload mass carried by flights using booster version “F9 v1.1”, which is equal to 2,928.4 kg.

First successful drone ship landing date

First_Date	Landing_Outcome
2016-04-08	Success (drone ship)

- The SQL query returns the date on which a successful landing on a drone ship was first achieved, which is April 8, 2016.

Successful ground pad landing with payload between 4,000 and 6,000

Booster_Version
F9 FT B1032.1
F9 B4 B1040.1
F9 B4 B1043.1

- The SQL query returns the list of booster versions used by flights successfully landing on a ground pad with a payload mass between 4,000 kg and 6,000 kg.

Total number of successful and failed mission outcomes

Mission Outcome	Total
Success	100
Mission Outcome	**Total**
Failure	1

- The SQL query returns the counts of mission successes and failures.

Boosters carried maximum payload

Booster_Version	PAYLOAD_MASS__KG_
F9 B5 B1048.4	15600
F9 B5 B1048.5	15600
F9 B5 B1049.4	15600
F9 B5 B1049.5	15600
F9 B5 B1049.7	15600
F9 B5 B1051.3	15600
F9 B5 B1051.4	15600
F9 B5 B1051.6	15600
F9 B5 B1056.4	15600
F9 B5 B1058.3	15600
F9 B5 B1060.2	15600
F9 B5 B1060.3	15600

- The SQL query returns the list of booster versions that were used in flights carrying the historical maximum payload mass.
- The maximum payload mass can be carried by flights with different booster versions. However, they may have different landing success rates.

2017 launch records

Yr	Mo	Landing_Outcome	Booster_Version	Launch_Site
2017	02	Success (ground pad)	F9 FT B1031.1	KSC LC-39A
2017	05	Success (ground pad)	F9 FT B1032.1	KSC LC-39A
2017	06	Success (ground pad)	F9 FT B1035.1	KSC LC-39A
2017	08	Success (ground pad)	F9 B4 B1039.1	KSC LC-39A
2017	09	Success (ground pad)	F9 B4 B1040.1	KSC LC-39A
2017	12	Success (ground pad)	F9 FT B1035.2	CCAFS SLC-40

- The SQL query returns the list of flights that successfully landed on a ground pad in 2017, wth their months of the year, booster versions, and launch sites.

Rank landing outcomes between 2010-06-04 and 2017-03-20

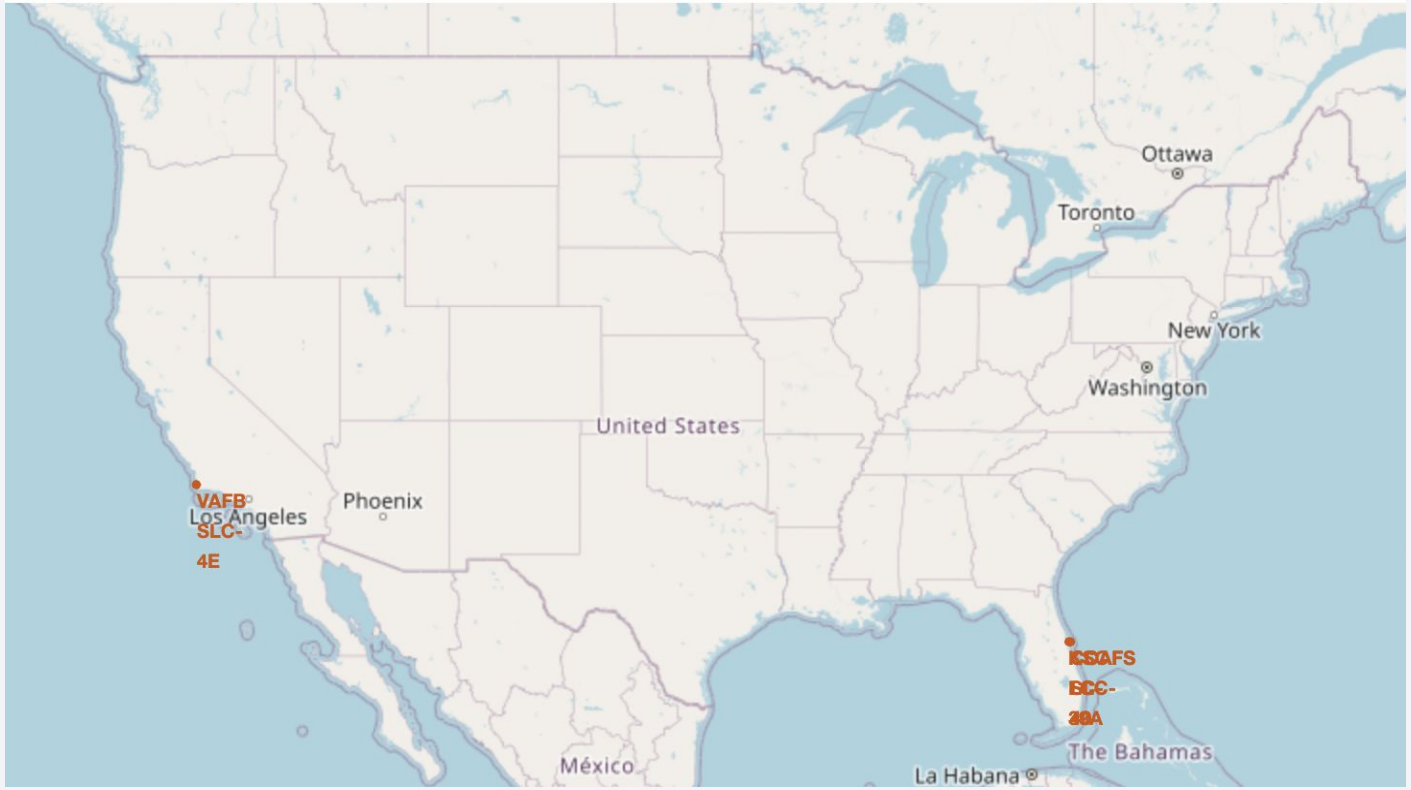
Landing_Outcome	Total
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

- The SQL query returns the counts of different landing outcomes between June 4, 2010 and March 20, 2017, in descending order.
- The landing outcomes before 2017 were mixed.

Section 3

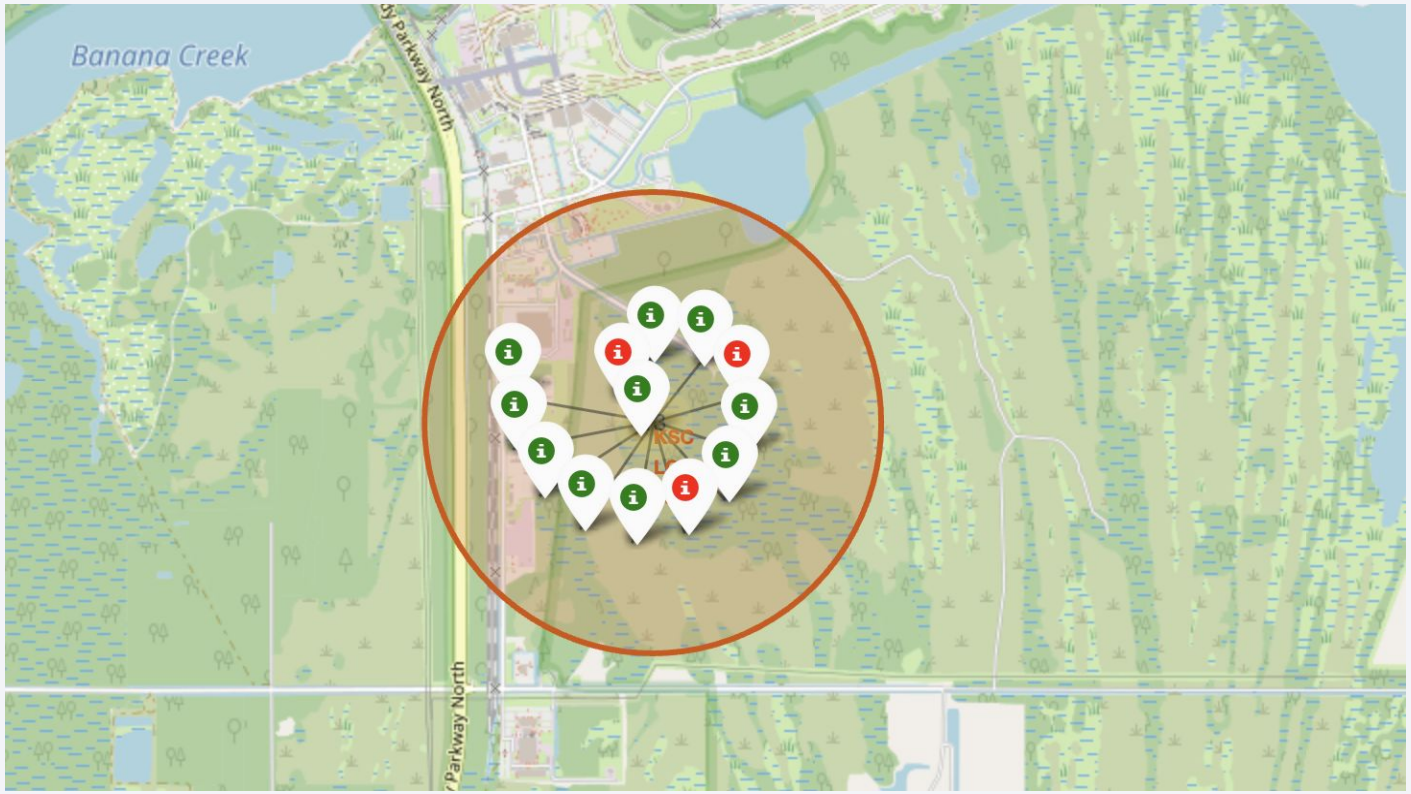
Launch Sites Proximities Analysis

SpaceX’s launch sites



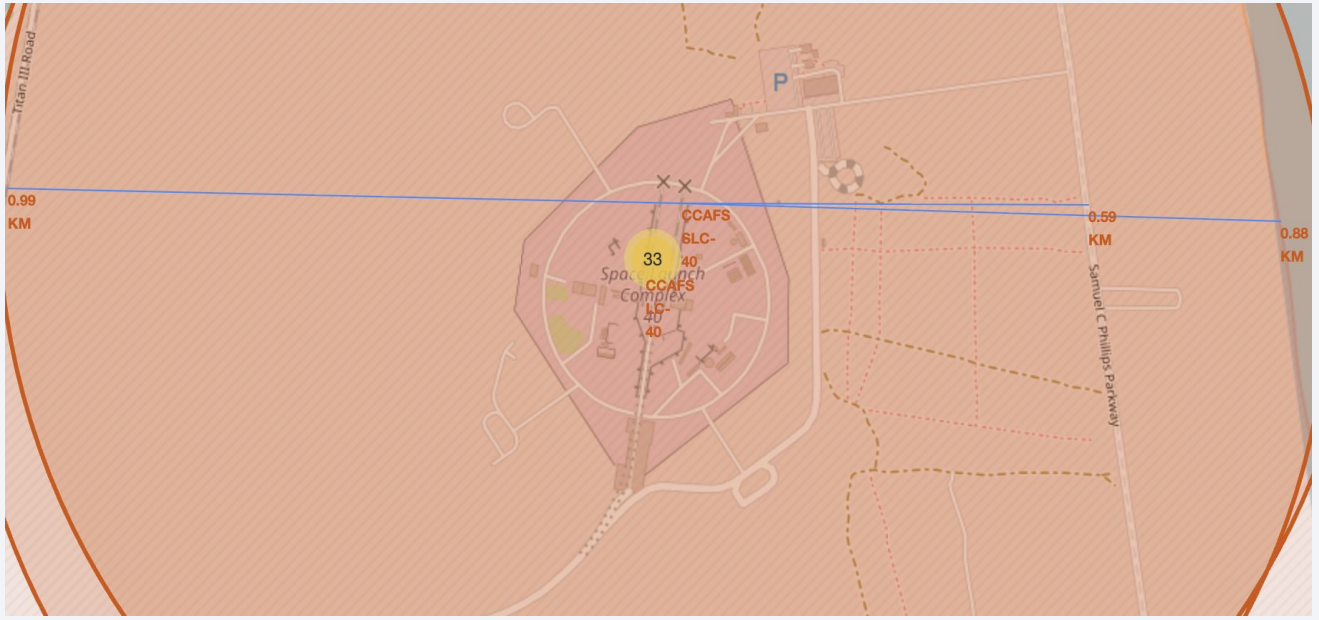
- All launch sites are located on the coastline, one on the west coast and the other three on the east coast.

KSC LC-39A's landing outcomes



- Launch site "KSC LC-39A" had had 13 flights, three of which ended up in landing failure.

CCAFS SLC-40's proximities

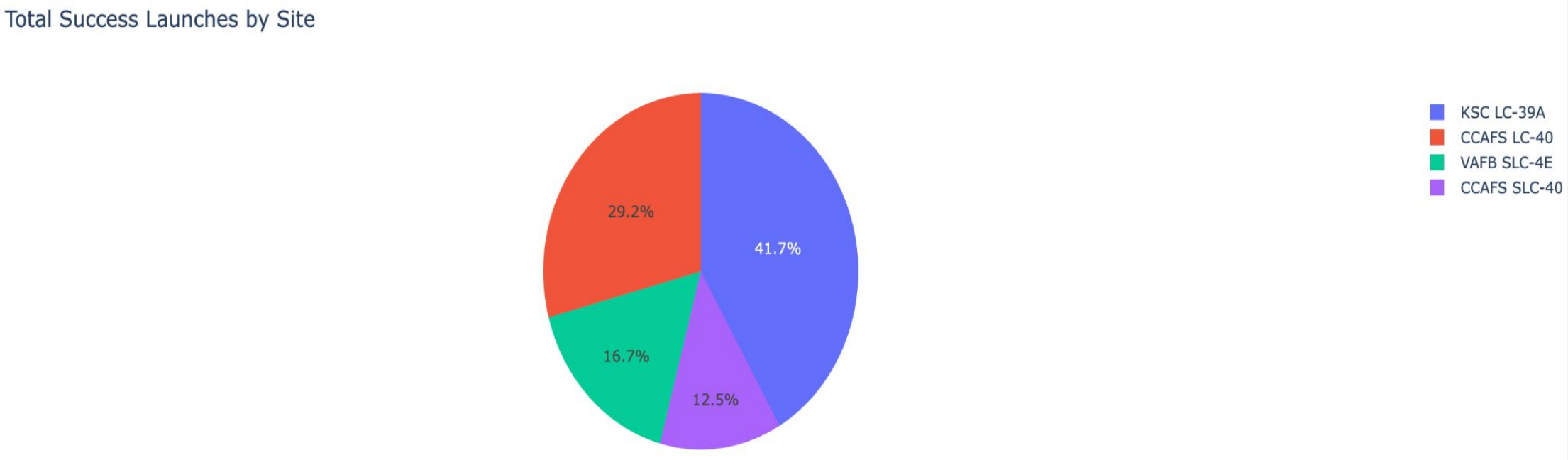


- The map measures distances from launch site "CCAFS SLC-40" to the nearest highway (0.59 km), coastline (0.88 km) and railroad (0.99 km).
- Close proximity to the highway and railroad indicates that access to transportation is highly desirable. Close proximity to the coastline corresponds with the option to attempt landing at sea, away from the population, and efficiently recover the equipment.
- A launch site's unique geographical characteristics may affect the possibility to attempt landing, thus affecting the landing success rate.

Section 4

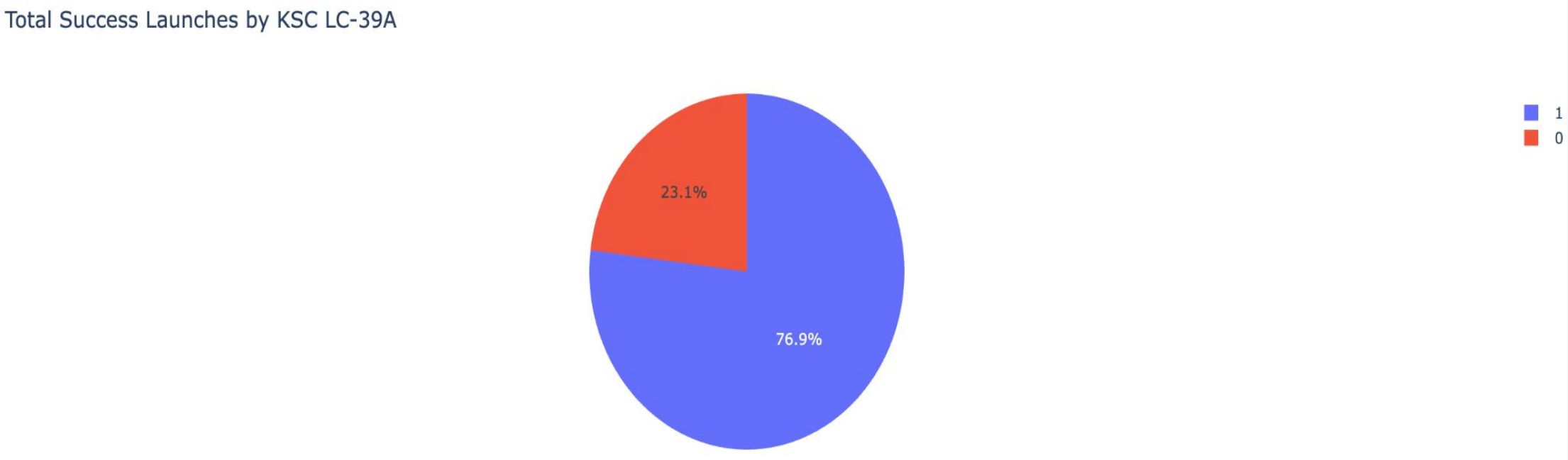
Build a Dashboard with Plotly Dash

Success launches' distribution by launch sites



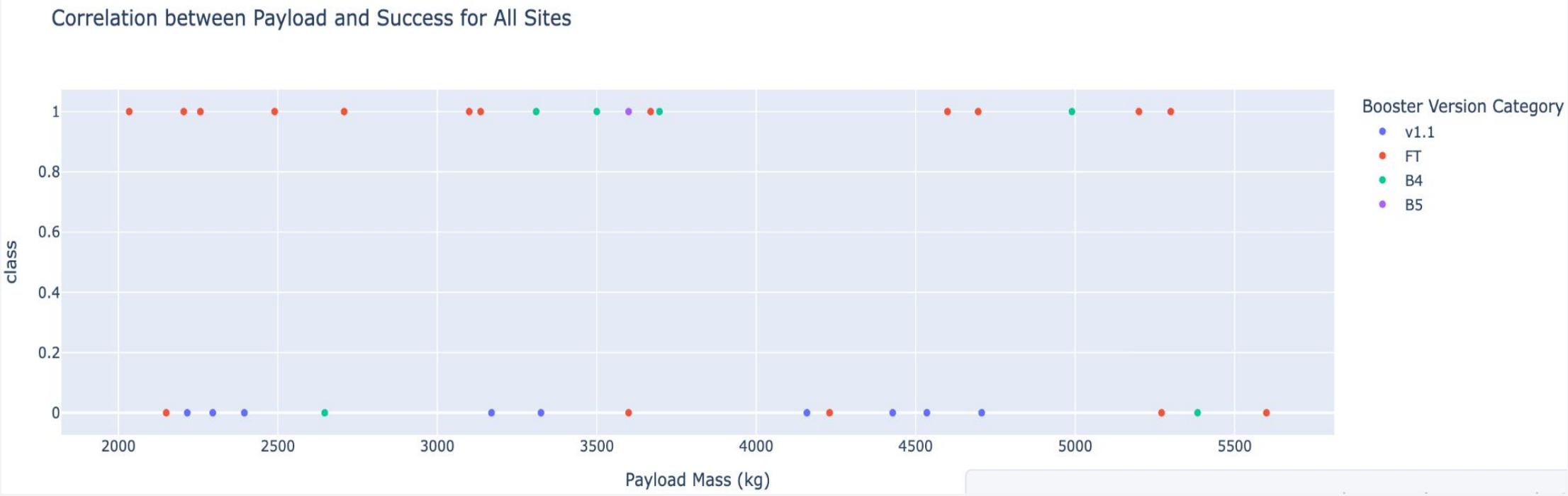
- The bar chart displays the distribution of successful landing outcomes w.r.t. launch site.
- However, the bar chart could be skewed by the launch sites' sample size.

KSC LC-39A's success/failure ratio



- The bar chart shows a landing success rate of 76.9% for launch site "KSC LC-39A", which is the highest among the launch sites.

Landing outcomes by booster version category

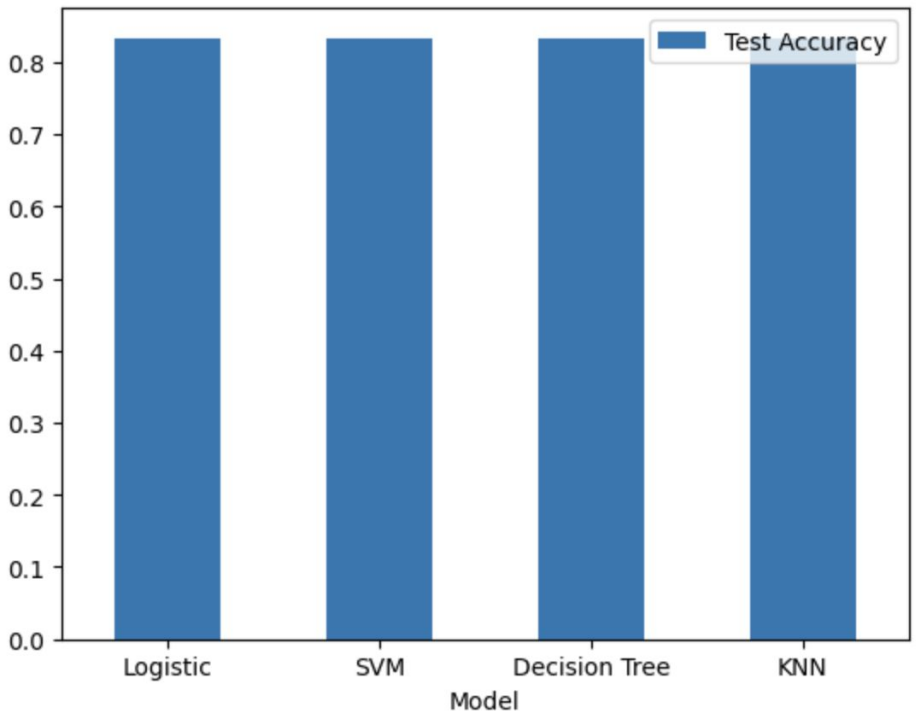


- The scatter plot shows, irrespective of launch site, the relationship between payload mass carried and landing outcome, using only flights carrying payload pass between 2,000 kg and 6,000 kg.
- Within this payload mass range, different booster version categories see different landing success rates, with version “FT” seemingly the best.

Section 5

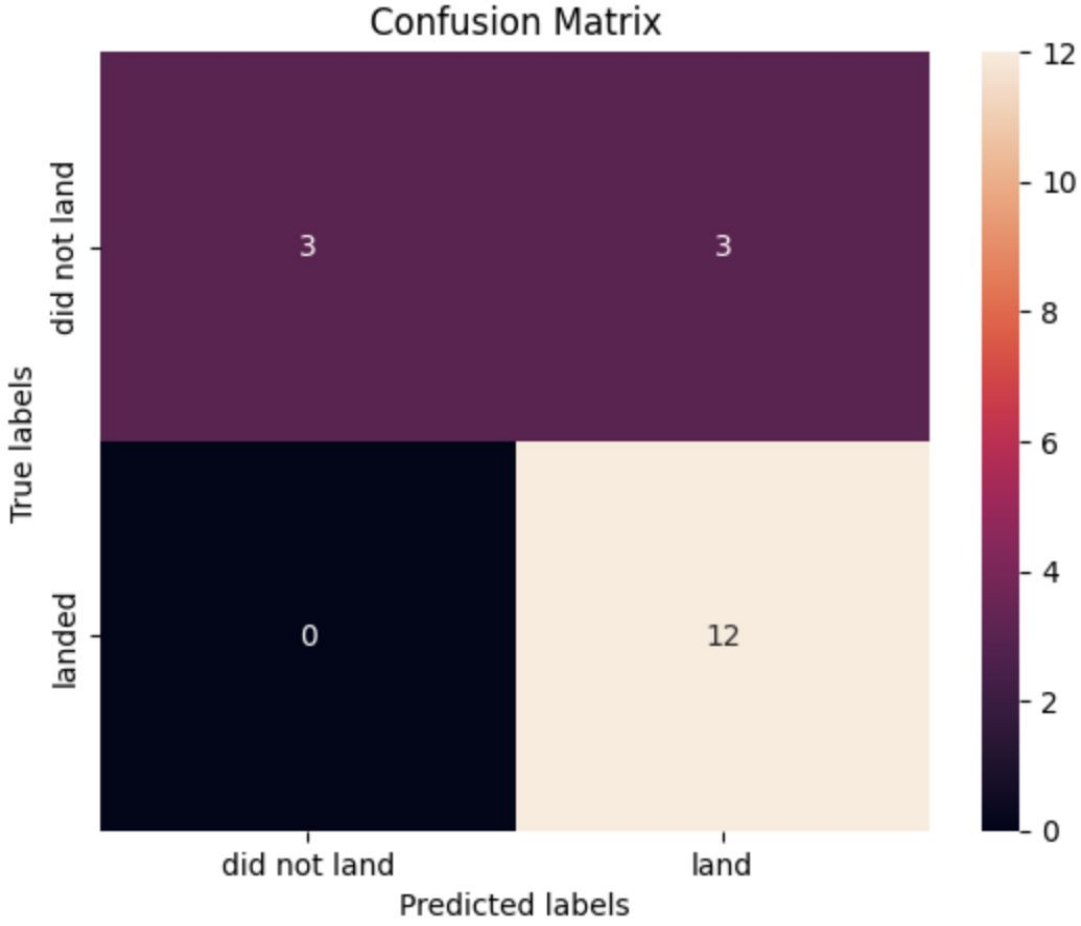
Predictive Analysis (Classification)

Classification accuracy



- CV specifications
  - Sample size = 90; size of the training set = 72; size of the testing set = 18
  - # of folds for CV = 10
  - Targets: “FlightNumber”, “PayloadMass”, “Flights”, “GridFins”, “Reused”, “Legs”, “Block”, “ReusedCount”, “Orbit”, “LaunchSite”, “LandingPad”, and “Serial”
  - Each ML algorithm’s HP values to train on (see Appendix)
- CV results
  - Each ML algorithm’s best mean “accuracy” score (see page 48)
  - Each ML algorithm’s best HP values (see Appendix)
  - Each best estimator’s “accuracy” score on the testing set: see the left

Confusion matrix



- As all four ML algorithms have the same confusion matrix, they do not only share the same “accuracy” score on the testing set, but also the same “precision”, “recall”, and “F1” scores.
- All four ML algorithms correctly predict all successful landing outcomes. However, they also have three false positives, which are of the error type costly here.

Conclusions

ML Algorithm	Best Mean Accuracy	Test Accuracy	Roc Score
Logistic	0.846429	0.833333	0.888889
SVM	0.848214	0.833333	0.958333
Decision Tree	0.873214	0.833333	0.868056
KNN	0.848214	0.833333	0.895833

- To distinguish the four ML algorithms, alternative measure ROC score is used, using SL's function "roc_auc_score".
  - SVM has the highest ROC score, indicating that it best distinguishes classes moving across probability thresholds and covers the largest area under the ROC curve.
- Although DT has the best mean "accuracy" score, it does not generalize better than others based on its best estimator's "accuracy" score on the testing set.
- Although SVM appears to be the winner, the caveat is that the sample size of 90 is small.

Appendix

- Additional CV specifications
  - LR's HP values to train on = {"C":[0.01,0.1,1],'penalty':['l2'],'solver':['lbfgs']}
  - SVM's HP values to train on = {'kernel':('linear','rbf','poly','rbf', 'sigmoid'),'C':np.logspace(-3,3,5),'gamma':np.logspace(-3,3,5)}
  - DT's HP values to train on = {'criterion':['gini','entropy'],'splitter':['best','random'],'max_depth':[2*n for n in range(1,10)],'max_features':['log2','sqrt'],'min_samples_leaf':[1,2,4],'min_samples_split':[2,5,10]}
  - KNN's HP values to train on = {'n_neighbors':[1,2,3,4,5,6,7,8,9,10],'algorithm':['auto','ball_tree','kd_tree','brute'],'p':[1,2]}
- Additional CV results
  - LR's best HP values = {'C':0.01,'penalty':'l2','solver':'lbfgs'}
  - SVM's best HP values = {'C':1.0,'gamma':0.03162277660168379,'kernel':'sigmoid'}
  - DT's best HP values = {'criterion':'gini','max_depth':12,'max_features':'log2','min_samples_leaf':1,'min_samples_split':5,'splitter':'best'}
  - KNN's best HP values = {'algorithm':'auto','n_neighbors':10,'p':1}
- Project's Github link: https://github.com/liwaikitbryan/IBM-DS-2026-01-28-Capstone

Thank you!